```
        minSdkVersion 15
        targetSdkVersion 23
    }

    sourceSets.main {
        manifest.srcFile 'AndroidManifest.xml'
        java.srcDirs = ['src']
        resources.srcDirs = ['src']
        res.srcDirs = ['res']
        jniLibs.srcDirs = ['libs']
    }
}
```

Modify the compiled SDK and build versions according to the toolset available. The project structure will look somewhat like what's shown in Figure 1.

4. tess-two needs Android NDK during runtime, which helps in embedding C and C++ code into your Android apps. This is needed for performance-intensive applications like OCR. Download the NDK if you don't have one. You could get it from *http://developer.android.com/ndk/downloads/index.html* and build the NDK by following the steps shown below.
   Set the following environment variables:
   - *NDK* as path to your downloaded and uncompressed NDK folder
   - *PATH* as *$NDK:$PATH*
   - *NDK_PROJECT_PATH* to your tess-two folder's path under *Libraries*, which was created in Step 2

   Now, you are all set to build the NDK by running the 'ndk-build' command from the downloaded NDK directory in Linux and Mac terminals or by using 'ndk-build.cmd' from the Windows command prompt. Finally, set the NDK path in Android Studio *(File->Project Structure->SDK Location->Android NDK Location).*

```
apply plugin: 'com.android.
library'

buildscript {
    repositories {
        mavenCentral()
    }
}

android {
    compileSdkVersion 23
    buildToolsVersion "23.0.3"

    defaultConfig {
        minSdkVersion 15
        targetSdkVersion 23
    }

    sourceSets.main {
```

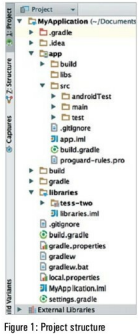

Figure 1: Project structure

```
        manifest.srcFile 'AndroidManifest.xml'
        java.srcDirs = ['src']
        resources.srcDirs = ['src']
        res.srcDirs = ['res']
        jniLibs.srcDirs = ['libs']
    }
}
```

5. Now that it is getting compiled as a separate library, we may have to include tess-two as a dependency for the actual app by adding the following line in the build script of the main project (the *build.gradle* under the app folder):

```
compile project(':libraries:tess-two')
```

Append this under the dependencies element of the app's build script (*build.gradle* under the app folder).

Finally, to compile both the app and the tess-two library, append the following line in *settings.gradle* (given below):

```
include ':libraries:tess-two'
```

Using the tess-two methods

With the above steps, Tesseract APIs should be compilable from the app. Before jumping onto invoking tess-two, you need to convert the image in *.png* or *.jpg* form to an Android bitmap. The following code snippet explains how to achieve this. Here, it is assumed that the image is stored in the internal storage (being referred by *Environment.getExternalStorageDirectory()*).

```
String path = new File(Environment.
getExternalStorageDirectory(), <image_name>).
getAbsolutePath();
FileInputStream fis = new FileInputStream(path);
Bitmap bitmap = BitmapFactory.decodeStream(fis);
```

One might experience several issues while obtaining the bitmap from an image. These include:

1. The *BitmapFactory.decodeStream()* might throw an out-of-memory exception or, in some cases, might not throw an exception but return a null bitmap. Make sure the image is not too big and RAM has enough space. A 500kb image typically takes nearly 5MB of RAM. Google provides suggestions for efficiently handling large bitmaps at *http://developer.android.com/training/displaying-bitmaps/load-bitmap.html*.
2. The bitmap could be null because there are no read/write permissions for the image storage. Make sure *AndroidManifest.xml* has these permissions set.
3. Double check the path from where the image is being read. It is always safe to use the *getAbsolutePath()* method for paths.